Francisco Redondo, ist1106117
Vasco Pereira, ist1103368
Group 16

Lab 2: Optimal Control in a Fragile Smart Home Markov Decision Processes and Policy Analysis

1.

In a Markov Decision Process, the best action at any given state s is found with the optimal policy ($\pi^*(s)$). We want to minimize the costs of energy production so the best action to take is the one that minimizes the sum of cost $C(s,a)$ and the future discount it will result in. Mathematically, this means finding the optimal value a, after computing the optimality function $V^*(s)$ for all the s states, using the Bellman Optimality Equation:

$$\pi^*(s) = \arg\min_{a \in \mathcal{A}} \left[C(s,a) + \gamma \sum_{s'} P(s'|s,a) V^*(s') \right]$$

Where:

- $C(s,a)$ is the immediate cost of taking action a in state s.$C(s,a)$ is the immediate cost of taking action a in state s.
- γ is the discount factor (usually between 0 and 1).γ is the discount factor (usually between 0 and 1).
- $P(s'|s,a)$ is the transition probability tensor you built in the Jupyter notebook.$P(s'|s,a)$ is the transition probability tensor you built in the Jupyter notebook.
- $V^*(s')$ is the optimal cost-to-go from the next state s'.$V^*(s')$ is the optimal cost-to-go from the next state s'.

In the case of having two different policies, say π_1 and π_2, one must evaluate their respective Value Functions, $V^{\pi_1}(s)$ and $V^{\pi_2}(s)$. The value function represents the expected cumulative cost of following a specific policy starting from state s.

This means that, policy π_1 is considered better than or equal to policy π_2 if its expected cumulative cost is lower or equal for every single state s in the state space:

$$V^{\pi_1}(s) \leq V^{\pi_2}(s) \quad \forall s \in \mathcal{S} \, V^{\pi_1}(s) \leq V^{\pi_2}(s) \quad \forall s \in \mathcal{S}$$

Policy π_1 is strictly better if the above condition holds, and there is at least one state where $V^{\pi_1}(s) < V^{\pi_2}(s)$.

2.

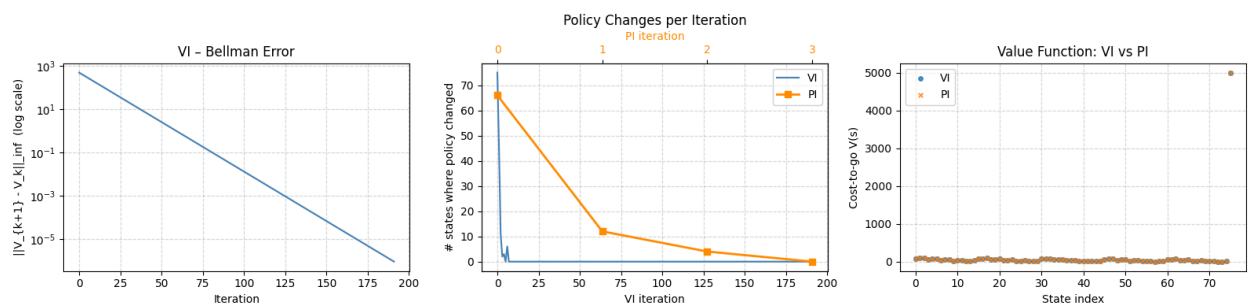

Fig. 1 - Policy Iteration (PI) vs Value Iteration (VI).

Policy Iteration (PI) converges in significantly fewer iterations than Value Iteration (VI). In the middle plot ("Policy Changes per Iteration"), you can see that the sequence for PI (the orange line with square markers) drops to 0 after just a handful of iterations (3 in this case). In contrast, the blue curve for VI stretches out much further, indicating that its policy takes much longer to stabilize. Furthermore, the first plot shows VI's Bellman error smoothly but slowly decaying over dozens or hundreds of iterations (depending on γ and ϵ).

This happens because VI only propagates information one step at a time per iteration. It slowly updates the value estimate ($V \leftarrow \min(C + \gamma PV)$), meaning the error shrinks by a factor of γ (0.9) at each step. On the other hand, PI evaluates the exact infinite-horizon cost of the current policy by solving a system of linear equations. By seeing the complete future consequences of a policy instantly, it makes massive jumps toward the optimal policy at each improvement step.

When it comes to advantages and disadvantages, Policy Iteration offers the distinct advantage of requiring incredibly few iterations to find the exact optimal policy, a behavior clearly seen in the rapid drop of the orange line in the middle plot. Additionally, it benefits from a clear, exact stopping condition by terminating precisely when the policy no longer changes. However, its

primary disadvantage lies in the policy evaluation step, which requires solving a system of linear equations of size $S \times S$ (where S is the number of states). This operation takes $O(S^3)$ time. While this is extremely fast for a small environment like this one where $S = 76$, scaling this to millions of states makes computing the inverse $(I - \gamma P^{\pi})^{-1}$ computationally intractable and excessively memory-intensive.

Conversely, Value Iteration is advantageous because it is easier to implement and each individual iteration is very fast to compute. Rather than solving complex linear systems, it relies entirely on basic matrix-vector multiplications taking $O(S^2 \times A)$ time, making the method far more scalable to larger state spaces. The main disadvantage is that it requires many more iterations to converge because the values only gradually creep toward the true solution, as visible in the long, drawn-out slope in the leftmost plot. Furthermore, establishing a stopping condition is trickier; the algorithm must wait until the maximum change (the Bellman error) drops below an arbitrary threshold, such as $\epsilon = 10^{-6}$. This requirement often leads to extra, computationally redundant sweeps that continue even after the actual optimal policy has already been identified.

3.

To evaluate the impact of costs, three distinct cost matrices were implemented:

- **Baseline Cost:** Represents the standard parameters defined in the lab guide (Grid cost = 10, Export revenue = -5, R_{fail}= 500).
- **High Export Cost:** Increases the revenue of exporting to the grid to -20, which models a scenario with highly lucrative feed-in tariffs.
- **Low Failure Cost:** Reduces R_{fail} to 20, which models a highly robust system where catastrophic failure is merely a minor inconvenience.

The requested cost matrices' parameters across every cost policy are discriminated in padilab2-group16.ipynb.

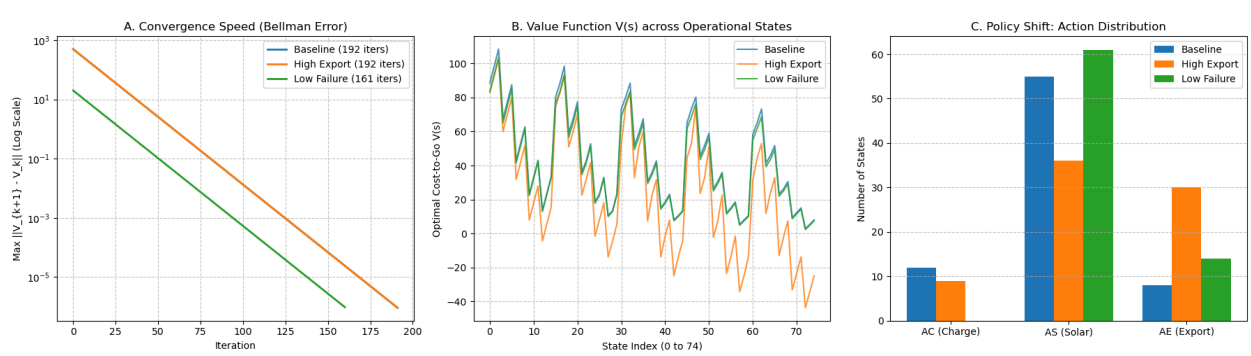

Fig. 2 - Cost impact.

Changing the cost function fundamentally alters both the agent's optimal policy and the convergence dynamics of the Value Iteration (VI) algorithm, as visualized in the generated plots:

As shown in Plot C Baseline (blue line), the agent relies heavily on Solar (AS) for ~55 states but actively chooses to Charge (AC) in about 12 critical states to ensure the battery doesn't empty out, avoiding the massive 500 penalty.

Regarding High Export (orange line) the policy shifts aggressively. Plot C shows a spike in the Export (AE) action (jumping to 30 states). This happens because of the -20 reward that is so lucrative, the agent is willing to hover near riskier battery levels to maximize profit. This is further proven by Plot B, where the orange line gets to negative values, indicating the system operates at a net profit.

About the scenario of Low Failure (green line), Plot C shows that the Charge (AC) action drops to *exactly zero*. Because the failure penalty (20) is barely higher than standard grid costs, the agent never wastes money on a safe grid charge. It relies entirely on Solar (AS) and Export (AE), risking failure because the penalty is negligible.

Plot A shows that altering the magnitude of the costs affects the number of iterations required for Value Iteration to converge. Both the Baseline and High Export scenarios overlap and take 192 iterations to converge, whereas the Low Failure scenario converges much faster, terminating in only 161 iterations. Lowering the maximum cost from 500 down to 30 allows the Bellman updates to propagate and reach the ϵ-convergence threshold more quickly.ϵ -convergence threshold more quickly.

(Note: Plot B only goes until state 74 since state 75 is a sink state, i.e. it always has the fixed cost value of R_{fail})

4.

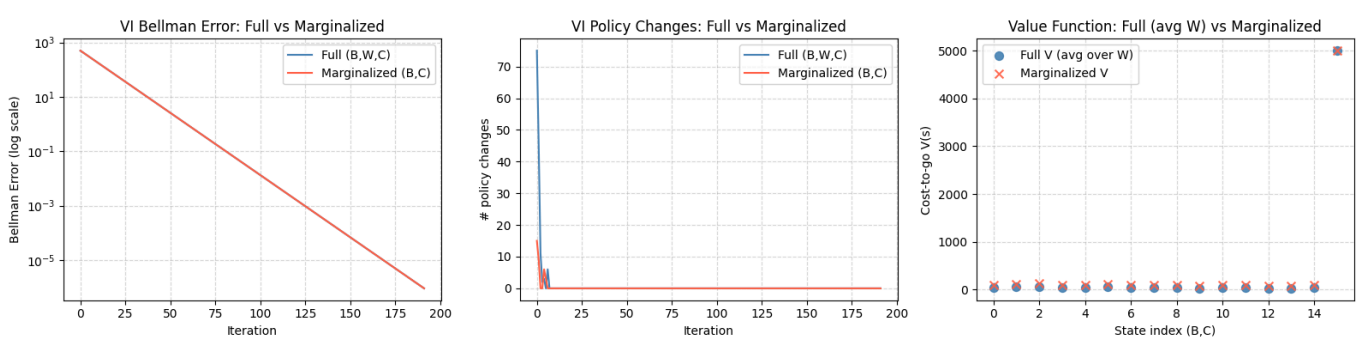

Figure 3 - VI between the Marginalized Model and the Full Model

```
─── Policy per (b,c): Full model (one per w=0..4) vs Marginalized ───
```

(b,c)	w=0	w=1	w=2	w=3	w=4	Marg
(0,0)	AC	AS	AS	AS	AS	AC
(0,1)	AC	AS	AS	AS	AS	AC
(0,2)	AC	AS	AS	AS	AS	AC
(1,0)	AC	AS	AS	AS	AS	AC
(1,1)	AC	AS	AS	AS	AS	AC
(1,2)	AC	AS	AS	AS	AS	AC
(2,0)	AC	AS	AS	AS	AS	AE
(2,1)	AC	AS	AS	AS	AS	AC
(2,2)	AC	AS	AS	AS	AS	AC
(3,0)	AE	AS	AS	AS	AE	AE
(3,1)	AC	AS	AS	AS	AS	AE
(3,2)	AC	AS	AS	AS	AS	AC
(4,0)	AE	AS	AE	AE	AE	AE
(4,1)	AE	AS	AS	AS	AE	AE
(4,2)	AC	AS	AS	AS	AS	AE

Including weather information in the state space is fundamentally important because it ensures the environment satisfies the Markov property. Without weather data, the process is non-Markovian: the probability that the "Active Solar" (AS) action successfully charges the battery depends entirely on the current, hidden weather.

While the inclusion of weather does not change how fast the algorithms converge, as both models require the exact same number of iterations to reach a solution, as seen in the identical Bellman error and policy-change curves, it completely changes what the policy converges to.

The policy table clearly demonstrates this shift. In the full model, the agent adapts its strategy dynamically: when it is sunny (w=3 or w=4), it reliably chooses AS because solar charging has a high success rate (80–95%), saving on grid costs. Conversely, when it is cloudy (w=0 or w=1), it switches to the deterministic "Active Charge" (AC) from the grid to guarantee battery levels, as solar becomes highly unreliable. The marginalized agent cannot make this distinction and is forced into a compromised, blended strategy. This single-action approach inevitably leads to inefficiencies, either taking risky AS actions during cloudy conditions or wasting money on AC when it is sunny and solar would have sufficed.

Although the plotted Value Functions for both models appear nearly identical, this visual is misleading. The near-perfect overlap only occurs because the full model's values are being averaged over all weather states, reflecting the same long-run average dynamics as the marginalized model. In practice, the full model holds a strict situational advantage. It can exploit specific states to minimize costs, such as safely taking "Action Empty" (AE) when the battery is full and the weather is sunny, or immediately paying for AC when the battery is low and it is cloudy. The marginalized model lacks this expressiveness, relying entirely on a static strategy regardless of the actual environment.